4. Accéder aux bases de données

Nitschke - Roka

Introduction

Connexion

Requêtes préparées sans données

Requêtes préparées avec données

- notations ‘?’ et ‘:’
- bindValue()
- bindParam()

Récupération des données

- foreach
- fetch(), fetchAll()
- bindColumn()

Jointures avec les JOIN (complément SQL)

Introduction

Besoin d'interagir avec une base de données

- Sites e-commerce (clients, produits, commandes, ...)
- Gestion du personnel d'une entreprise (congés, déplacements, ...)
- Gestion des comptes (banques)
- Gestion des notes, des stages, de l'emplacement aléatoire (département Info)
- ...

PHP => bases de données relationnelles (Oracle, MySQL, PostrgeSQL, **MariaDB**, ...)

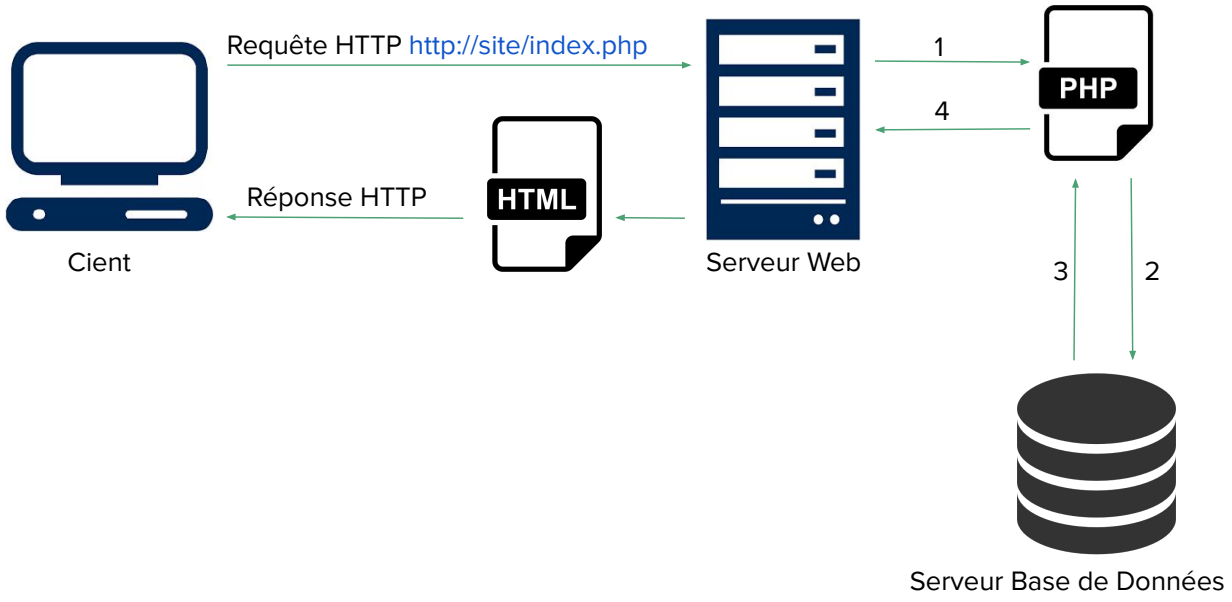
Introduction

Utilisation d'une base données en BUT1 :

- PHPMyAdmin
- MySQLWorkbench
- Saisies, requêtes "en dur", pas de formulaire "user-friendly"

Définition d'une interface utilisateur : PHP

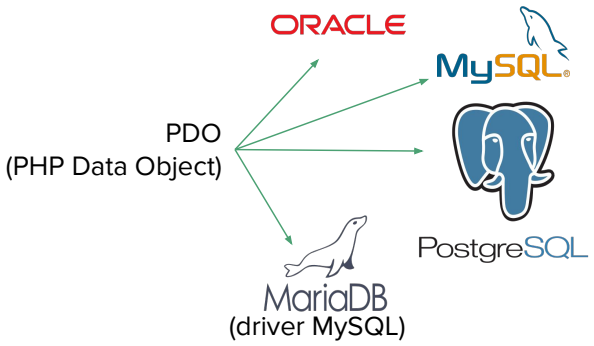
Introduction



Introduction

Accès à une base données

- à l'aide de PDO (PHP Data Object)
  - une extension qui définit une interface uniforme pour accéder aux différentes BdD



Introduction

- utilisation d'un driver de la BdD exposant ses fonctionnalités
- une couche d'abstraction de l'accès et non de la BdD :
  la syntaxe de la BdD doit être respectée
- 3 classes
  - PDO : connexion entre PHP et la BdD
  - PDOStatement : requête préparée et résultat associé
  - PDOException : exception levée par PDO

Connexion

Connexion

En créant un objet PDO :

```
<?php
try {
	$db = new PDO ('mysql:host=devbdd.iutmetz.univ-lorraine.fr;port=3306;dbname=toto3u_db',
	'toto3u_appli', 'password');
}
catch( Exception $exception ) {
	die($exception->getMessage());
}
?>
```

Pour un site en local (WAMP, LAMP, MAMP) :

```
	$db = new PDO ('mysql:host=localhost;dbname=nom_db', 'root', 'root');
```

Connexion

```
$objPdo = new PDO ('mysql:host=devbdd.iutmetz.univ-lorraine.fr;port=3306;dbname=toto3u_db',
                   'toto3u_appli', 'password');
```

```
<?php      //utilisation des variables pour les trois paramètres
    $source = 'mysql:host=devbdd.iutmetz.univ-lorraine.fr; dbname=toto3u_db',
    $utilisateur = 'toto3u_appli';
    $motDePasse =  'password';

    try {
        $db = new PDO ($source, $utilisateur, $motDePasse);
    }
    catch( Exception $exception ) {
        die($exception->getMessage());
    }
?>
```


Connexion

```
<?php                    //utilisation d'un tableau associatif pour l'ensemble des données de connexion
    $db_config['SGBD']          = 'mysql';
    $db_config['HOST']          = 'devbdd.iutmetz.univ-lorraine.fr';
    $db_config['DB_NAME']       = 'toto3u_db';
    $db_config['USER']          = 'toto3u_appli';
    $db_config['PASSWORD']      = 'password';
```

Attention : mot de passe en clair !

```
    try{
    $db = new PDO(  $db_config['SGBD'].':host='.$db_config['HOST'].';dbname='.$db_config['DB_NAME'],
                    $db_config['USER'], $db_config['PASSWORD'],
                    array(PDO::MYSQL_ATTR_INIT_COMMAND => 'SET NAMES utf8'));
                    // permet d'afficher les caractères utf8 si la BdD est définie en utf8 (accents...)
    unset($db_config);
    }
    catch( Exception $exception ) {
        die($exception->getMessage());
    }
?>
```

Exploitation de la base de données

Utilisation des requêtes préparées

Intérêts :

- sécurité des données, évite les injections SQL
- si une requête est utilisée plusieurs fois, pas besoin de la répéter
- pour des requêtes paramétrés

On utilise les méthodes :

- prepare() : retourne un objet PDOStatement
- bindValue() : lie des valeurs ou des variables à l'objet
  bindParam() : lie des variables à l'objet
- execute() : exécute la requête

Utilisation des requêtes préparées

Utilisation de méthode **prepare()**

Requête de sélection, sans données :

```
$requete = ' SELECT * FROM table ';
$res = $db->prepare($requete);      // on prépare la requête d'affichage
$res -> execute();                  // on exécute la requête
```

Requête avec données (sélection, insertion) :

dans la requête SQL, on remplace les données par

- soit des points d'interrogation '?'
- soit par des noms de paramètre précédés par un double point ':'

Utilisation des requêtes préparées (bindValue)

on remplace les données par des points d'interrogation '?' :

```
$requete = 'INSERT INTO table (attr1, attr2) VALUES (?,?);

$res  = $db->prepare($requete);          // on prépare la requête d'insertion de donnée

$res -> bindValue(1,  'valeur1');        // on associe des valeurs ou des variables aux paramètres
$res -> bindValue(2, 'valeur2');         //                                ( bindValue() )

$res -> execute();                       // on exécute la requête
```

on remplace les données par des noms de paramètre précédés par un double point ':' :

```
$requete = 'INSERT INTO table (attr1, attr2) VALUES (:param1,:param2);

$res  = $db->prepare($requete);          // on prépare la requête d'insertion de donnée

$res -> bindValue('param1',   $var1);    // on associe une variable au param1, $var1 doit avoir une valeur
$res -> bindValue('param2',  'valeur2');  // on associe une valeur au param2

$res -> execute();
```


Utilisation des requêtes préparées (exemple)

Exemple (schéma de relation) :

personne (id_pers, nom_pers, prenom_pers)

id_pers	nom_pers	prenom_pers
1	SKYWALKER	Luke
2	SOLO	Han

Exemples de requête :

1. Liste des personnes ($listPers)
2. Insérer une personne dans la table personne ($insertPers)

Traitement en PHP :

```
$listPers       = ' SELECT * FROM personne ORDER BY nom_pers ';
$insertPers     = ' INSERT INTO pers (nom_pers, prenom_pers) VALUES(:nom, :prenom) ';

$res  = $db->prepare($insertPers);       // on prépare la requête d'insertion de donnée
$res -> bindValue('nom', 'ORGANA');      // on associe des valeurs aux paramètres – bindValue()
$res -> bindValue('prenom', 'Leia');
$res -> execute();                       // la ligne a été insérée
$res  = $db->prepare($listPers);         // on prépare la requête de récupération des données
$res -> execute();                       // on récupère le résultat de la requête dans $res (une table)
```

Utilisation des requêtes préparées (bindParam)

```
$listPers        = ' SELECT * FROM personne ORDER BY nom_pers ';
$insertPers      = ' INSERT INTO pers (nom_pers, prenom_pers) VALUES(:nom, :prenom) ';

$res  = $db->prepare($insertPers);          // on prépare la requête d'insertion de donnée

$res -> bindParam('nom',      $nom);        // on associe des variables aux paramètres
$res -> bindParam('prenom', $prenom);
$nom       = "YODA";                        // affectation des valeurs aux variables
$prenom   = "maître";

$res -> execute();                          // la ligne a été insérée
$res  = $db->prepare($listPers);            // on prépare la requête de récupération des données
$res -> execute();                          // on récupère le résultat de la requête dans $req (table)
```


Exécution directe d'une requête (sans bindValue/bindParam)

```
$insertPers = ' INSERT INTO pers (id_pers, nom_pers, prenom_pers) VALUES (:id, :nom, :prenom) ';

$res  = $db->prepare($insertPers);

$nom       = "SKYWALKER";
$prenom   = "Luke";

$res -> execute([':id'          => null,        // valeur 'null' pour les champs auto-incrémentés
                 ':nom'         => $nom,
                 ':prenom'      => $prenom]);

$res -> execute(array(':id'          => null,
                      ':nom'         => $nom,
                      ':prenom'      => $prenom));
```

On peut aussi ne pas mentionner le champ AI :

```
$insertPers = INSERT INTO pers (nom_pers, prenom_pers) VALUES (:nom, :prenom) ';
...
$res -> execute([':nom'         => $nom,
                 ':prenom'      => $prenom]);
```

Extraction des données retournées

```
$res = $db->prepare($listPers);        // on prépare la requête de récupération des données
$res -> execute();
```

- Parcours classique d'un tableau associatif (**foreach**)

```
foreach ($res as $personne){
        echo $personne['prenom_pers'].' '.$personne['nom_pers'].'<br /> ';
}
```

- Méthode **fetch**

  $res->fetch() récupère les données ligne par ligne

```
while ($personne = $res->fetch()){
        echo $personne['prenom_pers'].' '.$personne['nom_pers'].'<br /> ';
}
```


Extraction des données retournées

```
$res = $db->prepare($listPers);        // on prépare la requête de récupération des données
$res -> execute();                     // on exécute la requête (résultat : type PDOStatement)
```

- Méthode **fetch** avec la constante PDO::FETCH_OBJ

  $res->fetch(PDO::FETCH_OBJ) récupère les données sous forme d'objet

```
while ($personne = $res->fetch(PDO::FETCH_OBJ)){
        echo $personne->prenom_pers.' '.$personne->nom_pers.'<br /> ';
}
```

Extraction des données retournées

```
$res = $db->prepare($listPers);        // on prépare la requête de récupération des données
$res -> execute();                     // on exécute la requête
```

- Méthode **fetchAll**

  $res->fetchAll() récupère toutes les données sous forme de tableau

```
$allPers = $res-> fetchAll();
foreach ($allPers as $personne) {
        echo $personne['nom_pers'].' '.$personne['prenom_pers'];
}
```

public function fetchAll(int $mode = PDO::FETCH_DEFAULT): array

Valeurs possibles pour $mode :

PDO::FETCH_BOTH	:	valeur par défaut, renvoie un 'array', indexé par à la fois par des entiers et les clés
PDO::FETCH_ASSOC	:	renvoie un 'array', indexé par les clés
PDO::FETCH_CLASS	:	renvoie une instance de classe : nom d'une colonne => propriété

Extraction des données retournées (bindColumn())

- lie les colonnes à des variables

```
$listPers          = ' SELECT * FROM personne ORDER BY nom_pers ';

$res  = $db->prepare($listPers);
$res -> execute();
$res -> bindColumn(1, $id);
$res -> bindColumn(2, $nom);
$res -> bindColumn(3, $prenom);
        ou     $res -> bindColumn('id_pers', $id);
               $res -> bindColumn('nom_pers', $nom);
               $res -> bindColumn('prenom_pers', $prenom);

while ($res->fetch()){
               echo $id.' '.$prenom.' '.$nom.'<br /> ';
}
```

Libération de la ressource après utilisation : unset($res);

Redirection vers une page : header('location: nomPage.php');

Complément SQL (les JOIN)

Jointures avec JOIN

Intérêt : optimiser les requêtes

jointure classique :

```
FROM    R1, R2, R3
WHERE   R1.x = R2.x
AND     R2.y = R3.y
```

FROM : produit cartésien
WHERE : sélection

Dans le schéma relationnel

R1 (x, ...)
R2 (x, y, ...)
R3 (y, ...)

inconvénient : beaucoup de tables et beaucoup de lignes, sélection sur une table volumineuse

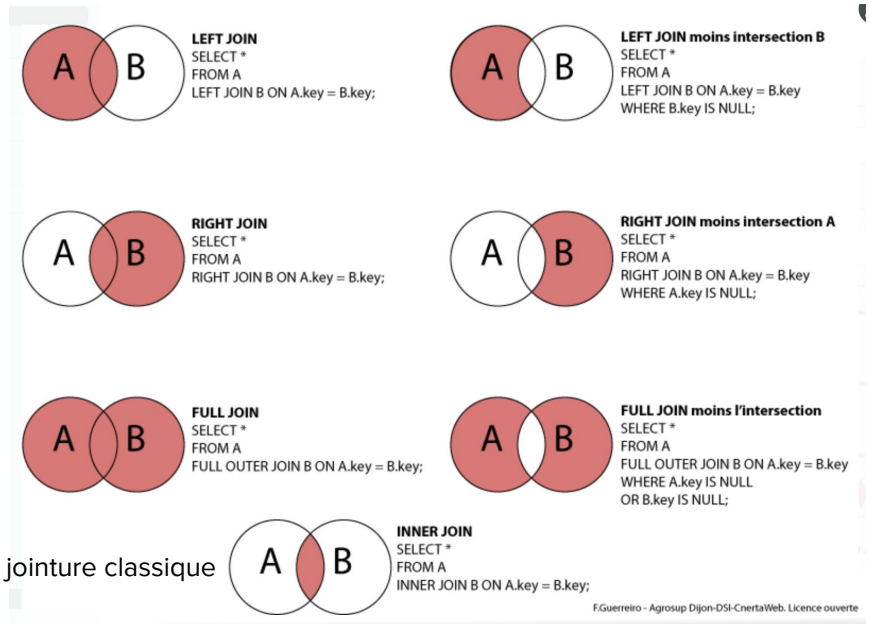
jointure en utilisant JOIN :

```
FROM R1
JOIN R2 ON R1.x = R2.x
JOIN R3 ON R2.y = R3.y
```

le nombre de n-uplets est minimal après chaque jointure

Les différents types de JOIN

jointure classique
+ “cas 0”



différence A\B
(not IN)

différence B\A
(not IN)

union

jointure classique

Les différents types de JOIN

- INNER JOIN (ou JOIN) : jointure classique sur attributs communs

  FROM R1 JOIN R2 ON R1.x = R2.x

- NATURAL JOIN : jointure classique, sans préciser les attributs sur lesquels les jointures se font (uniquement si les noms sont les mêmes)

  R1(<u>x</u>, y#) R2(<u>y</u>, z)
  FROM R1 NATURAL JOIN R2 (FROM R1, R2 WHERE R1.y=R2.y)

  R1(<u>x, y</u>, z#) R2(<u>z</u>, y#)
  FROM R1 NATURAL JOIN R2 (FROM R1, R2 WHERE R1.y=R2.y AND R1.z=R2.z)

Les différents types de JOIN

➢ Exemple LEFT JOIN :

client

id_cl	nom_cl
1	Bougdira
2	Ginoux
3	Nitschke
4	Roka

commande

id_com	id_cl	date_com
1	1	2022-10-05
2	2	2022-10-06

```
SELECT *
FROM client
LEFT JOIN commande ON client.id_cl=commande.id_cl
```

id_cl	nom_cl	id_com	id_cl	date_com
1	Bougdira	1	1	2022-10-05
2	Ginoux	2	2	2022-10-06
3	Nitschke	*NULL*	*NULL*	*NULL*
4	Roka	*NULL*	*NULL*	*NULL*

... WHERE commande.id_cl IS NULL

id_cl	nom_cl	id_com	id_cl	date_com
3	Nitschke	*NULL*	*NULL*	*NULL*
4	Roka	*NULL*	*NULL*	*NULL*

Les différents types de JOIN

“Nombre de commandes par client”

Requête avec jointure classique :

```
SELECT      client.id_cl, nom_cl, COUNT(*) "nb commandes"
FROM        client, commande
WHERE       client.id_cl = commande.id_cl
GROUP BY    client.id_cl, nom_cl
UNION
SELECT      id_cl, nom_cl, 0
FROM        client
WHERE       id_cl NOT IN (
                SELECT id_cl
                FROM commande)
```

Résultat de la requête :

id_cl	nom_cl	nb commandes
1	Bougdira	1
2	Ginoux	1
3	Nitschke	0
4	Roka	0

Requête avec LEFT JOIN :

```
SELECT      client.id_cl, nom_cl, COUNT(id_com)  "nb commandes"
FROM        client
LEFT JOIN   commande ON client.id_cl = commande.id_cl
GROUP BY    client.id_cl, nom_cl
```